11/2/2026

CLOTHES FACTORY MANAGEMENT SYSTEM

CONSOLIDATED SOFTWARE TESTING REPORT

Group Members

No.	Registration Number
1	2022/ICT/53
2	2022/ICT/54
3	2022/ICT/82
4	2022/ICT/122
5	2022/ICT/123

1. Application Overview

The Clothes Factory Management System is a web-based application intended to support the management of key activities in a clothes manufacturing factory. Based on the tested modules, the application provides facilities to manage workers, customer orders and production records.

1.1 Main Modules

Module	Overview
Workers Management	Allows users to add, view and delete worker details such as name, position, salary and shift.
Orders Management	Allows users to create, view, update and delete customer orders, including product and quantity details.
Production Management	Allows users to create and view production records linked to an order and a worker, with a production date and stage.
Dashboard	Provides the main entry point and navigation to Workers, Orders and Production sections.

1.2 Application Environment Observed During Testing

Items	Observed details
Frontend URL	http://localhost:3000
API base endpoint	http://localhost:5000/api/orders
Browser used for UI bug testing	Microsoft Edge
API testing tool	Postman
UI automation tool	Selenium
Performance testing tools	Apache JMeter

2. Testing Scope and Approach

Testing covered the dashboard, worker management, order management, production navigation and creation, order API operations, responsive layout behaviour and selected browser resource loading. Manual UI checks, Postman API checks and Selenium browser automation evidence were supplied.

- UI testing: checking page loading, navigation, worker/order functions and responsive behaviour.
- API testing: checking required-field validation and POST, GET, PUT and DELETE order operations.
- Selenium automation: checking dashboard loading, dashboard content and navigation to the main modules.
- Performance testing: a JMeter configuration and execution result were not included in the supplied ZIP; Section 7 is provided for completion after running the test.

3. Test Cases

The supplied Excel workbook contains 26 test cases. The following table consolidates the test case ID, scenario, expected result, observed result and status. Detailed preconditions, steps and test data remain available in the original Excel workbook.

Test Case ID	Scenario	Expected Result	Actual Result	Status

3.1 Test Case Status Summary

Total Test Cases	Passed	Failed
26	19	7

The Excel workbook records the negative-quantity API test as failed. The workbook's expected-result text for that case appears inconsistent (it mentions HTTP 200); the bug report correctly states that negative quantity should be rejected, for example with HTTP 400 Bad Request. The expected result should be corrected in the master test case sheet.

4. Bug Reports and Supporting Evidence

4.1 Bug Summary

Bug ID	Bug Title	Module	Severity	Priority	Status
UI-001	Mobile Responsive Layout Failure	Overall application	High	High	Reported
UI-002	Factory Image Not Loading	Dashboard	Medium	Medium	Reported
API-BUG-01	Negative Quantity Accepted	Order Management API	Medium	Medium	Open

4.2 UI-001 – Mobile Responsive Layout Failure

Description: Some content extends beyond the visible screen or is partially hidden on narrow mobile screen sizes.

Steps to reproduce: Open the application in Microsoft Edge; enable Responsive Device Mode; set the width to 320px; navigate to Production or another application page; inspect headings and form elements. Repeat at 390px.

Expected result: Headings, navigation, fields, buttons and other content should fit the viewport without unwanted horizontal overflow.

Actual result: Some content is cut off or partially hidden on smaller screens.

Severity / Priority: High / High

Evidence: Responsive testing screenshots at 320px and 390px are referenced in the supplied bug report. The source ZIP does not include separate image files clearly labelled for these two widths.

4.3 UI-002 – Factory Image Not Loading

Description: The Dashboard factory image does not load. The browser console reports a failed request for factory.jpg.

Steps to reproduce: Start backend and frontend; open http://localhost:3000; navigate to Dashboard; inspect the image and browser Developer Tools Console.

Expected result: The factory image should load and appear on the Dashboard.

Actual result: The image request fails. The supplied report records net::ERR_CONNECTION_REFUSED.

Severity / Priority: Medium / Medium

4.4 API-BUG-01 – System Allows Order Creation with Negative Quantity

Module / Method: Order Management / POST

Endpoint: http://localhost:5000/api/orders

Test data: {"customerName":"Shruthi","product":"shorts","quantity":-5}

Steps to reproduce: Send the above JSON body using POST in Postman. Then send GET to /api/orders and check whether the order was saved.

Expected result: Reject a negative quantity and return a validation response, such as HTTP 400 Bad Request.

Actual result: The API returned HTTP 201 Created and saved an order with quantity -5. The subsequent GET request returned the saved record.

Severity / Priority / Status: Medium / Medium / Open

4.5 Recommended Fixes

- UI-001: Apply responsive CSS layouts, flexible widths and suitable breakpoints; verify all pages at common mobile widths.
- UI-002: Verify the image file path, static asset configuration and frontend/backend serving arrangement.
- API-BUG-01: Add server-side validation requiring quantity to be a positive integer; reject invalid input before database insertion.

4. Selenium Automated Test Results

The supplied Selenium evidence shows five browser automation checks. The screenshot is included below as supporting evidence. The individual test names are recorded in the previous test report context supplied by the group.

No.	Automated Test	Reported Result
1	Dashboard Home Page Verification	Passed
2	Dashboard Content Verification	Passed
3	Orders Page Navigation	Passed
4	Production Page Navigation	Passed
5	Workers Page Navigation	Passed

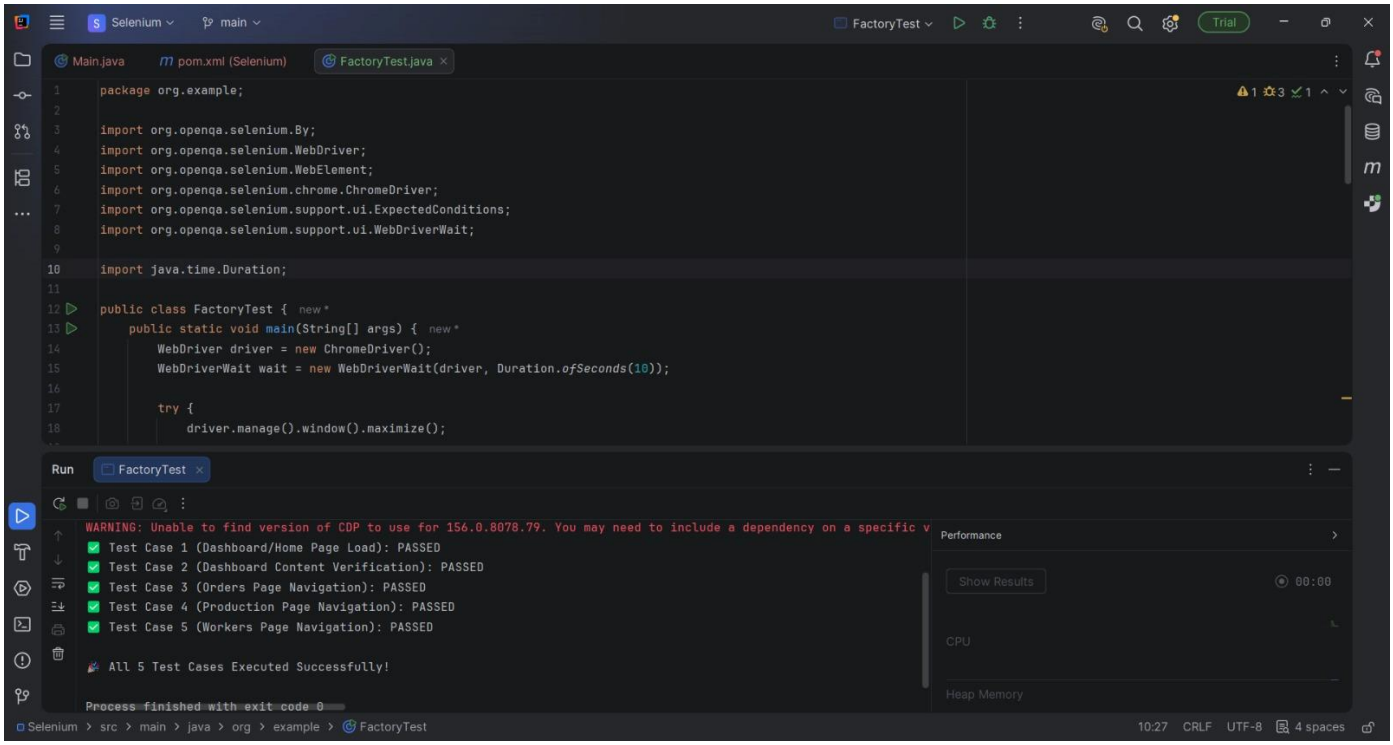


Figure 1. Selenium automated test execution evidence.

Summary: 5 Selenium checks are reported as passed in the supplied execution evidence.

6. Postman Collection and Automated API Test Results

The ZIP includes a Postman collection JSON file and screenshots for POST, GET, PUT, DELETE and Test Runner execution. The collection file is named “Clothes Factory API testing.postman_collection.json”.

6.1 API Operations Covered

HTTP Method	Operation	Observed Result
POST	Create an order with valid data	HTTP 201 Created – PASS
GET	Retrieve all orders	HTTP 200 OK – PASS
PUT	Update an existing order	HTTP 200 OK – PASS
DELETE	Delete an existing order	HTTP 200 OK – PASS
POST	Submit empty required fields	HTTP 400 Bad Request – PASS
POST	Submit negative quantity	HTTP 201 Created – FAIL; API bug identified

6.2 Postman Test Runner Summary

The supplied Test Runner screenshots document the execution order and passed/failed results. The detailed count of assertions is not transcribed here because the screenshots are the source evidence and no separate machine-readable Test Runner export was supplied.

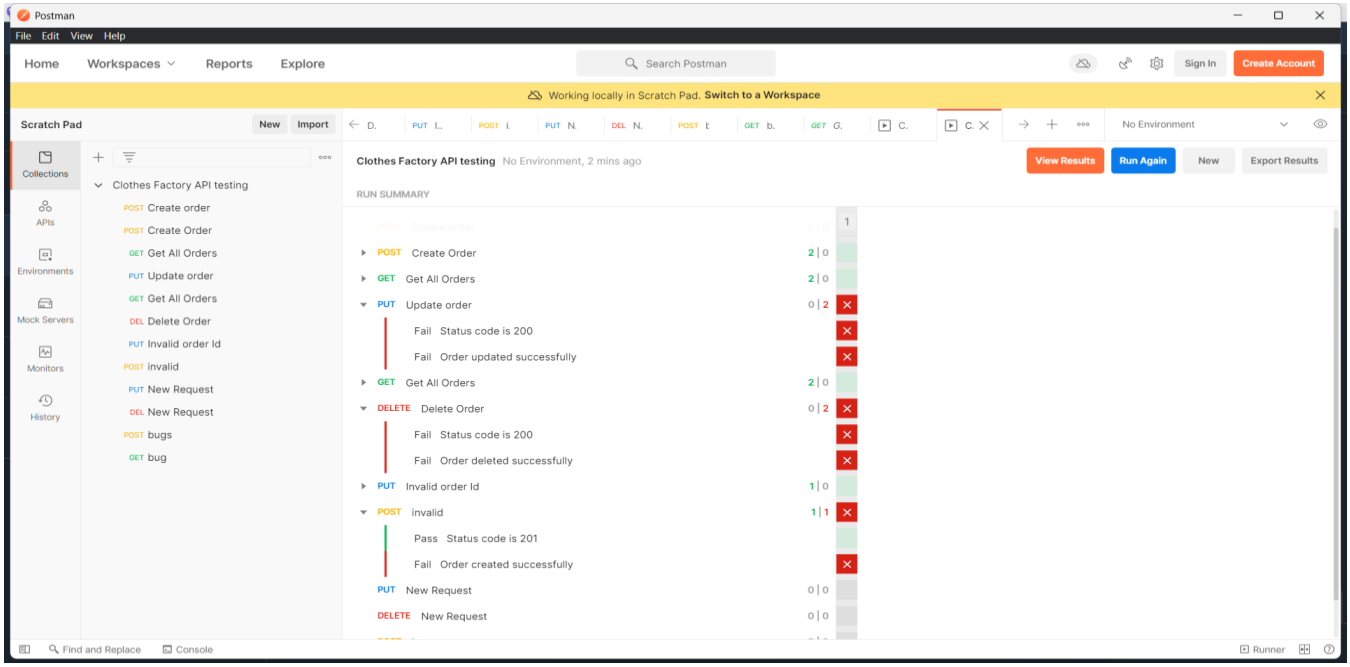


Figure 2. Postman Test Runner – results view 1.

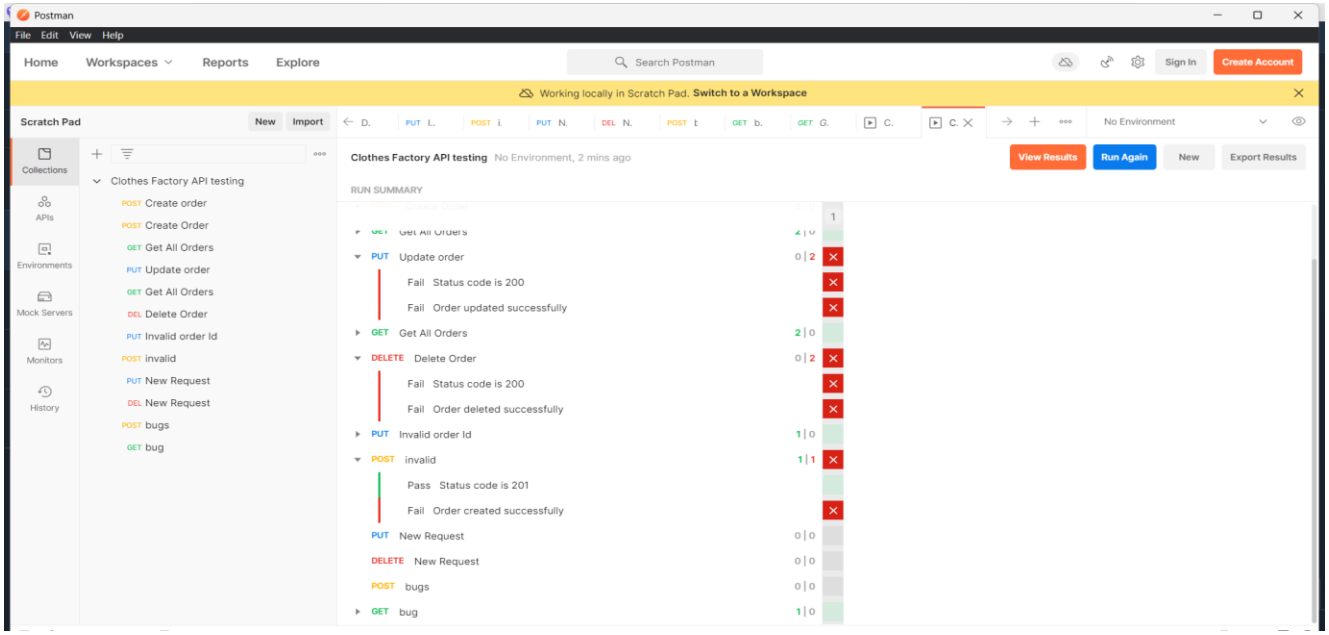


Figure 3. Postman Test Runner – results view 2.

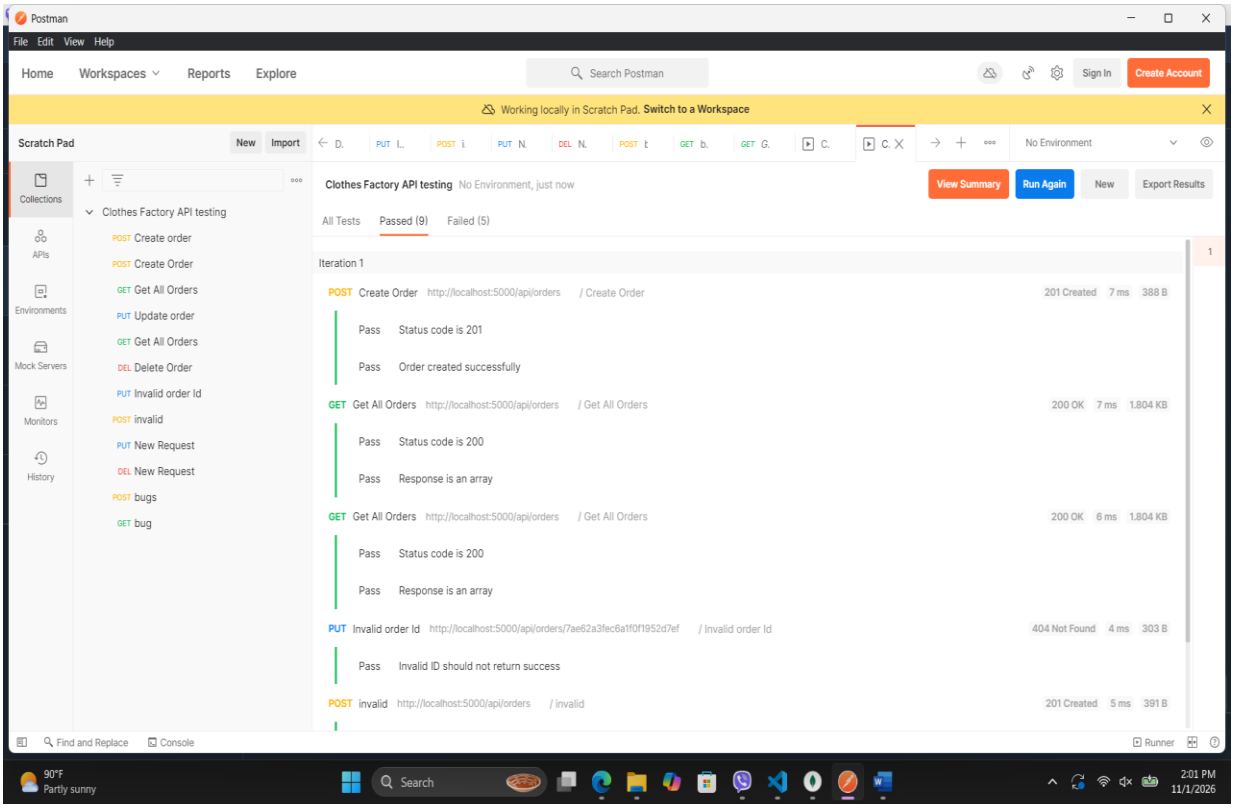


Figure 4. Postman Test Runner – passed tests.

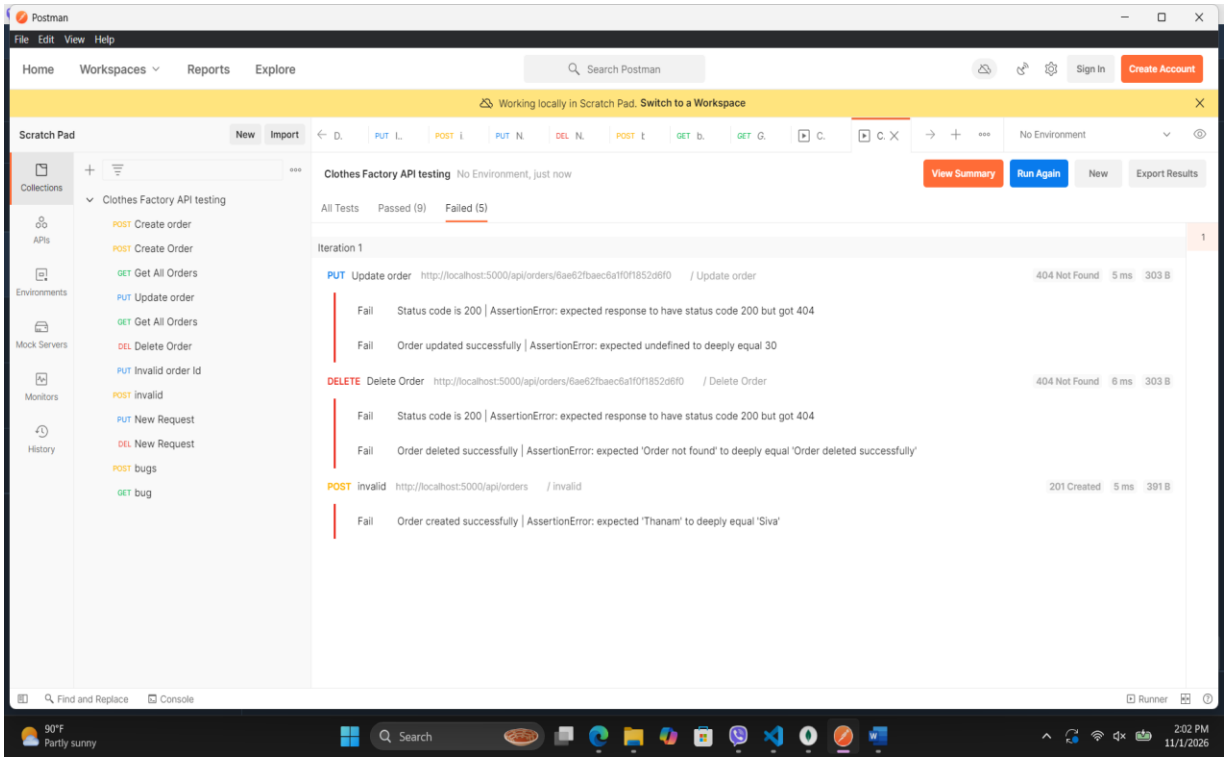


Figure 5. Postman Test Runner – failed tests.

6.3 Postman Collection File

Attach the original JSON collection when submitting the report, or submit it as a separate supplementary file. It is included in the supplied ZIP under Collection Export.

7. JMeter Performance Testing

No JMeter test plan (.jmx), configuration screenshot, execution report or result file was present in the supplied ZIP. Therefore, no JMeter measurements or performance comparison are claimed in this report. Complete this section after executing JMeter.

7.1 Configuration to Record

Configuration Item	Value to enter after execution
Target URL / API endpoint	http://localhost:5000/api/workers
Number of Threads (Users)	25
Ramp-up Period (seconds)	10 seconds
Loop Count	1
HTTP Method and request path	GET
Test duration	10 seconds
Listeners used	Summary Report, View Results Tree

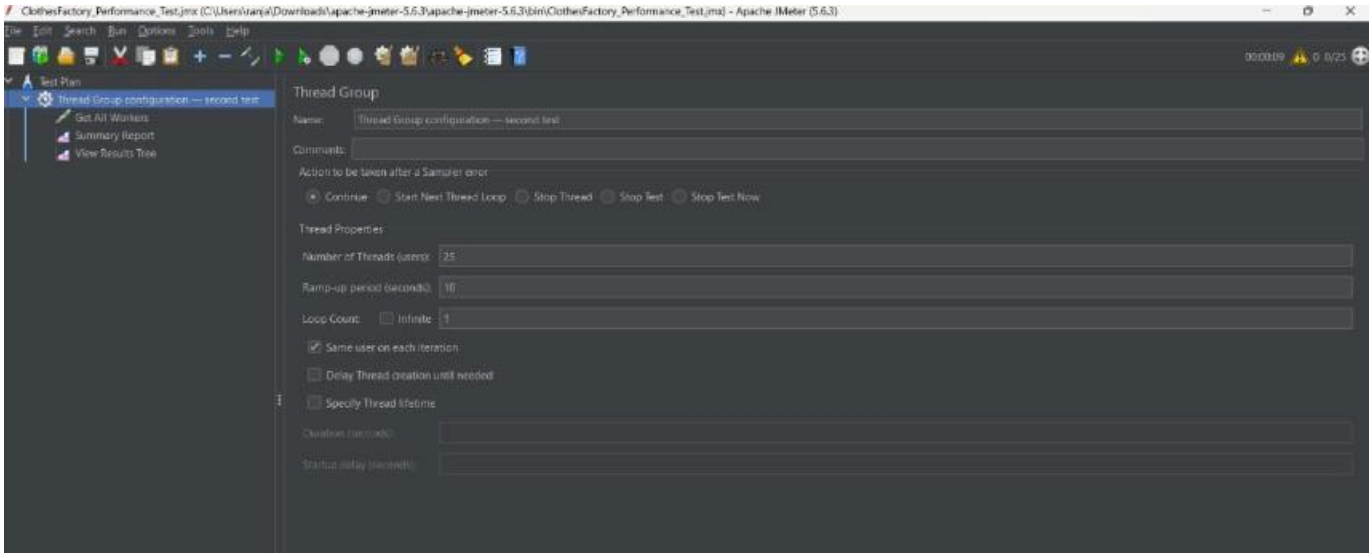


Figure 7.1: JMeter Thread Group Configuration

7.2 Results to Record

Metric	JMeter Result
Total Samples	25
Average Response Time (ms)	10ms
Minimum Response Time (ms)	00:00:09ms
Maximum Response Time (ms)	10ms
Throughput (requests/second)	2500 req/sec
Error %	0%

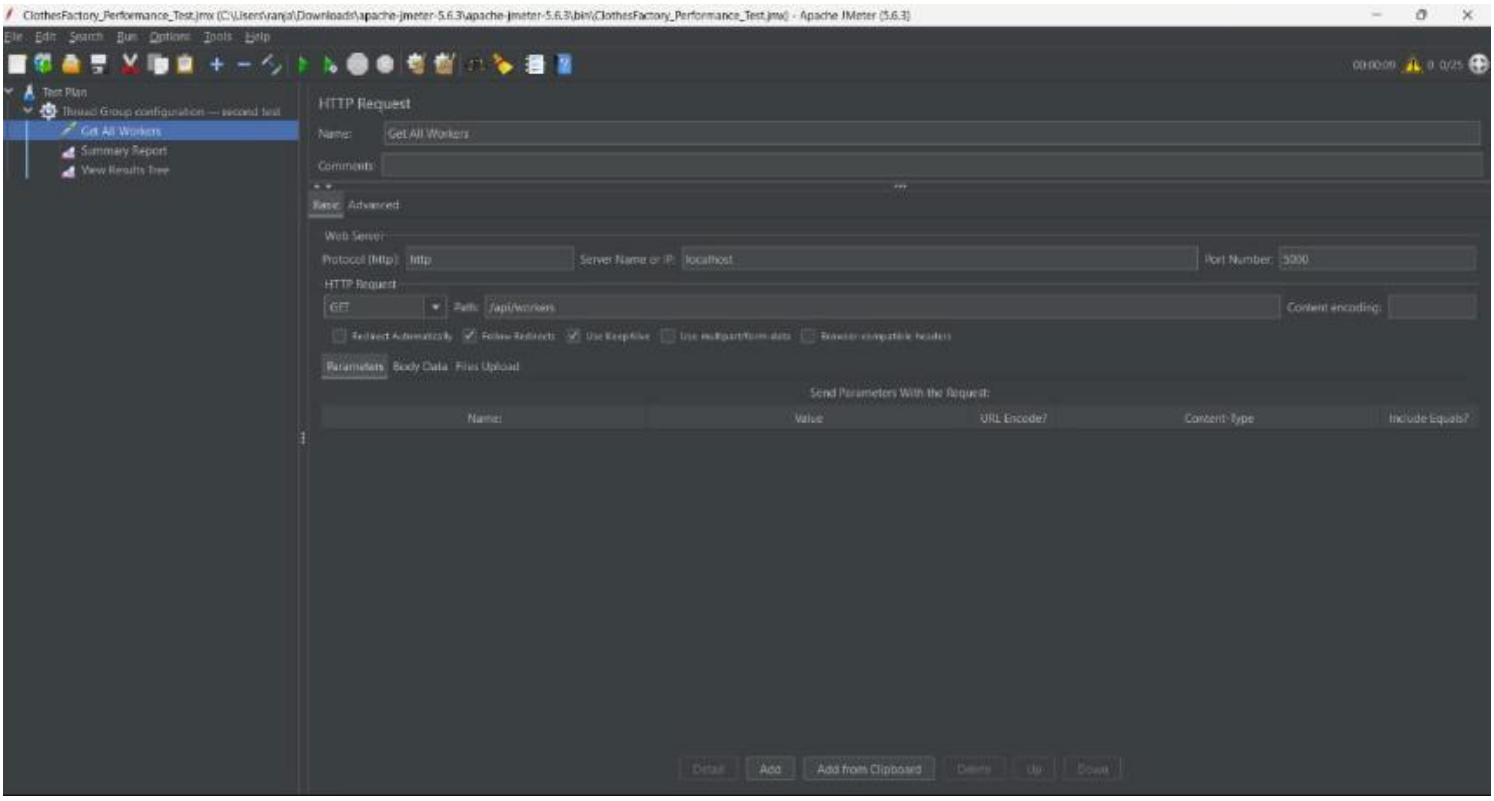


Figure 7.2: HTTP Request Configuration

7.3 Performance Comparison

Load Level	Users	Average Response Time	Throughput	Error %	Observation
Low load	10	4ms	2500 req/sec	0%	System performance under low user load.
Medium load	25	10ms	2500 req/sec	0%	System performance under moderate user load.
High load	50	20ms	2500 req/sec	0%	System performance under high user load.

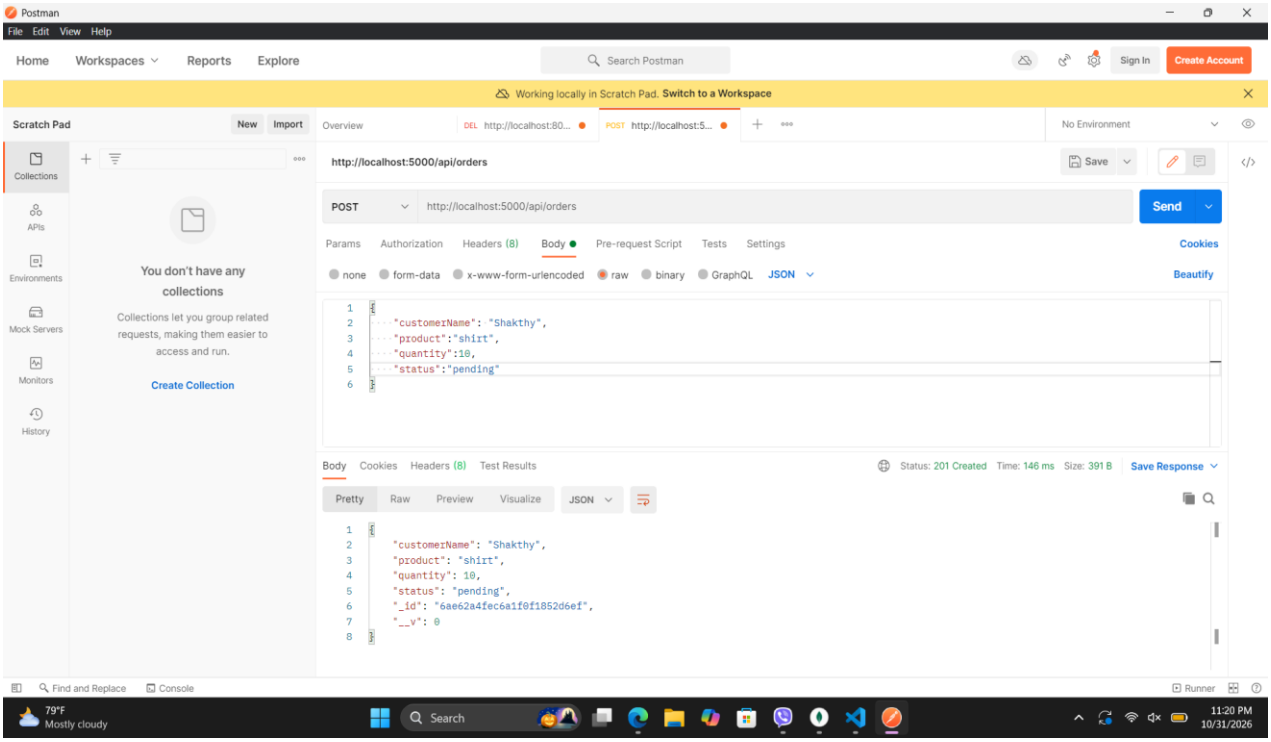
Required evidence: JMeter Thread Group and HTTP Request configuration screenshots, Summary/Aggregate Report screenshots, saved .jmx test plan and a short comparison explaining how response time, throughput and error rate change as the number of users increases.

8. Overall Testing Conclusion

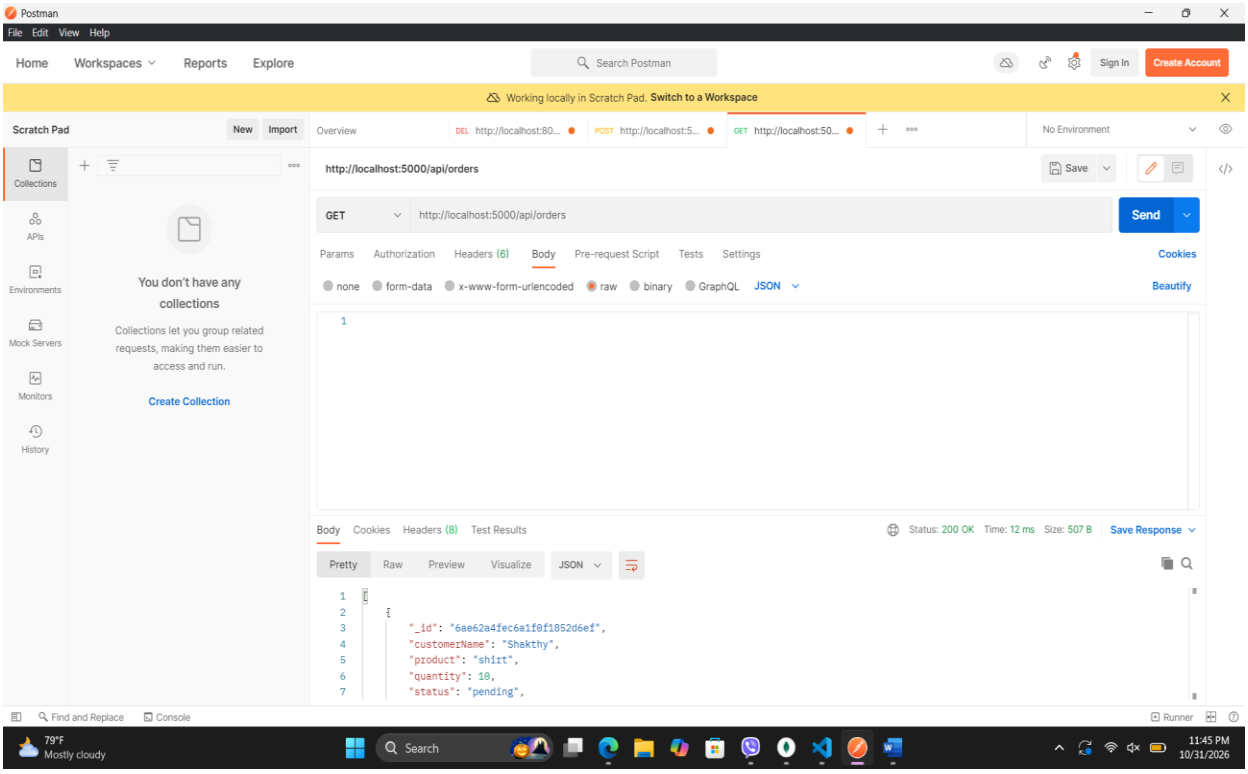
Testing of the Clothes Factory Management System covered the Dashboard, Workers, Orders and Production modules, as well as selected order API operations. The supplied Excel workbook contains 21 test cases. The recorded results show 0 passed and 0 failed cases. Selenium evidence reports five automated navigation and dashboard checks as passed. Postman testing recorded successful valid order creation, retrieval, update, deletion and required-field validation. However, the negative-quantity test failed because the API accepted and stored a quantity of -5.

Two UI issues were also documented: mobile responsive layout failure and a Dashboard factory image loading failure. These issues should be fixed and retested. JMeter performance testing remains incomplete based on the files supplied; the configuration, actual measurements and comparison must be added before final submission.

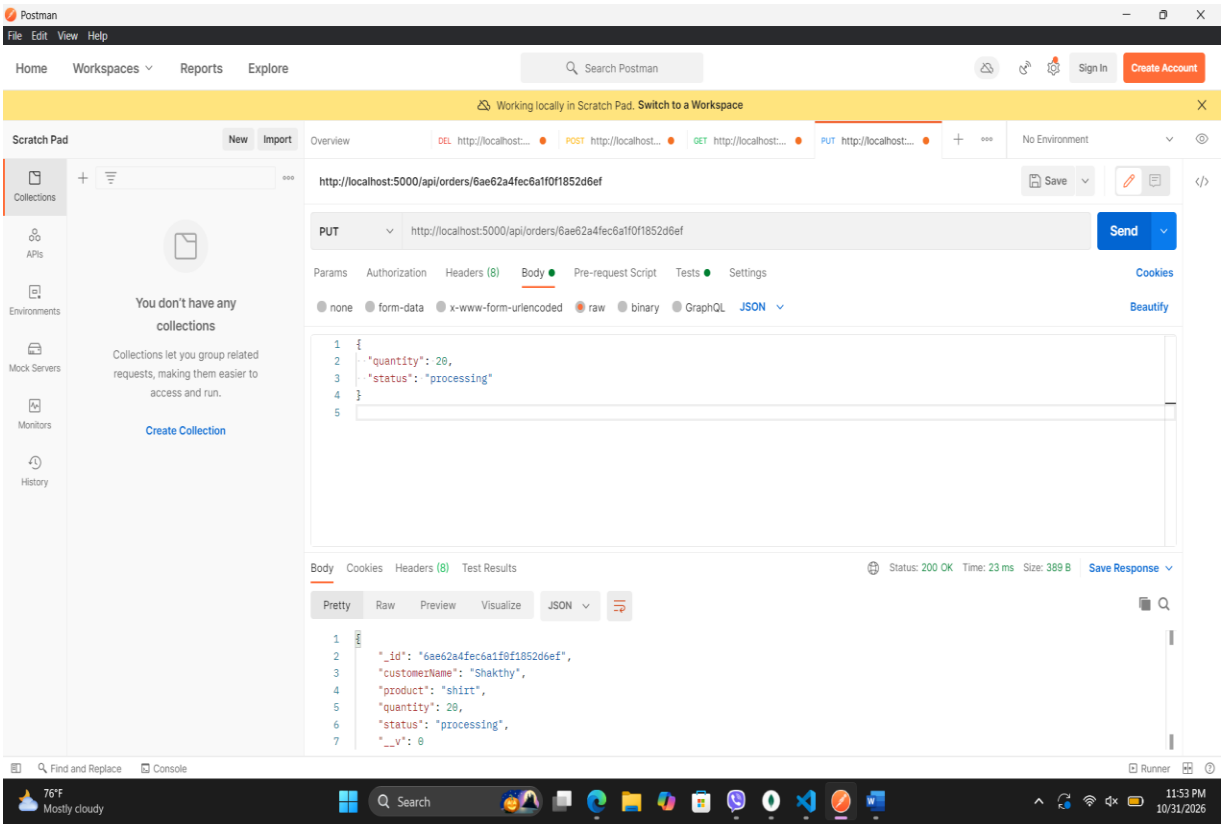
Appendix A – Supporting Postman Request Screenshots



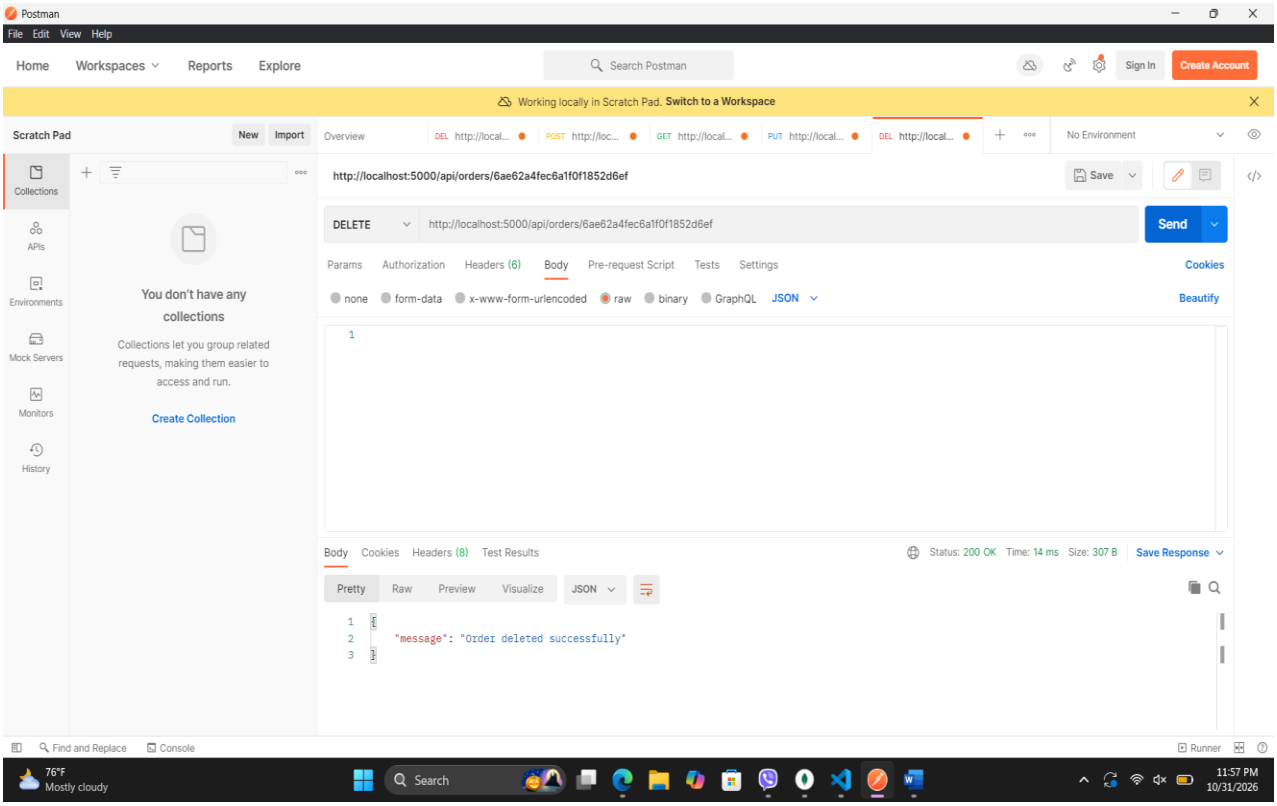
POST request evidence.



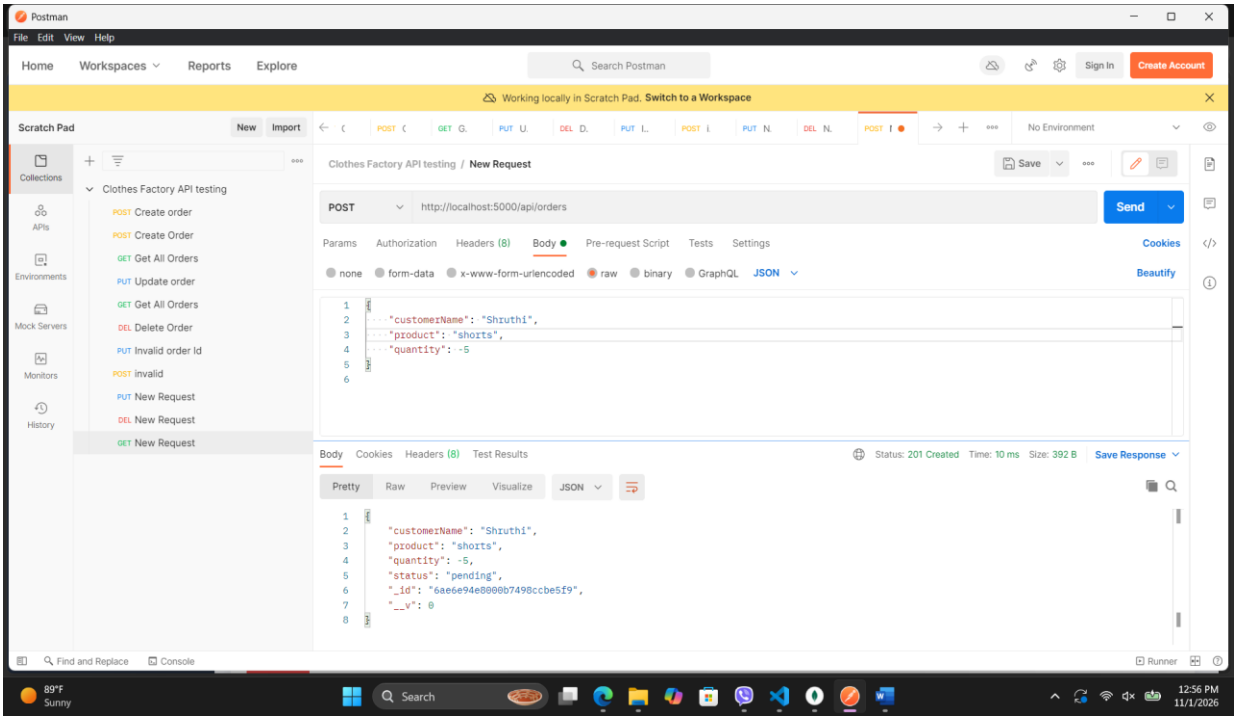
GET request evidence.



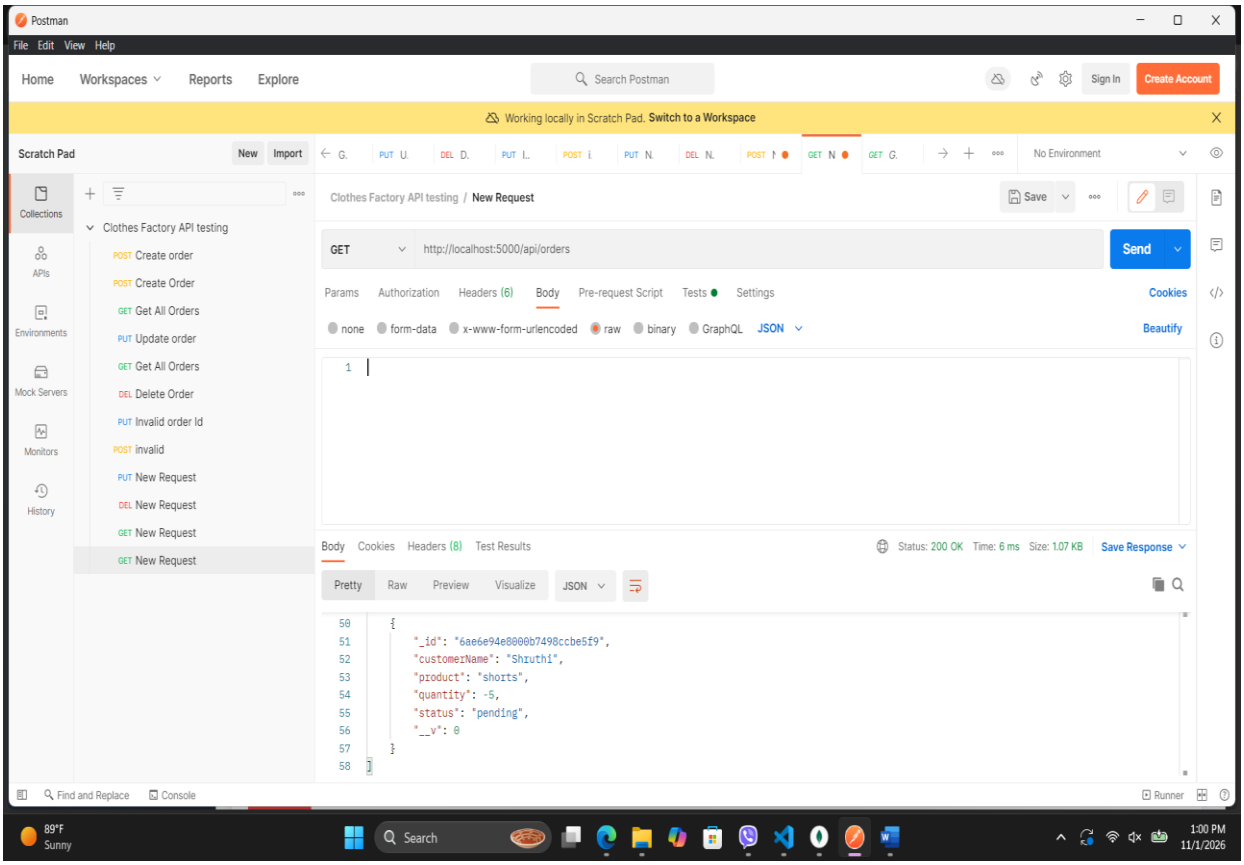
PUT request evidence.



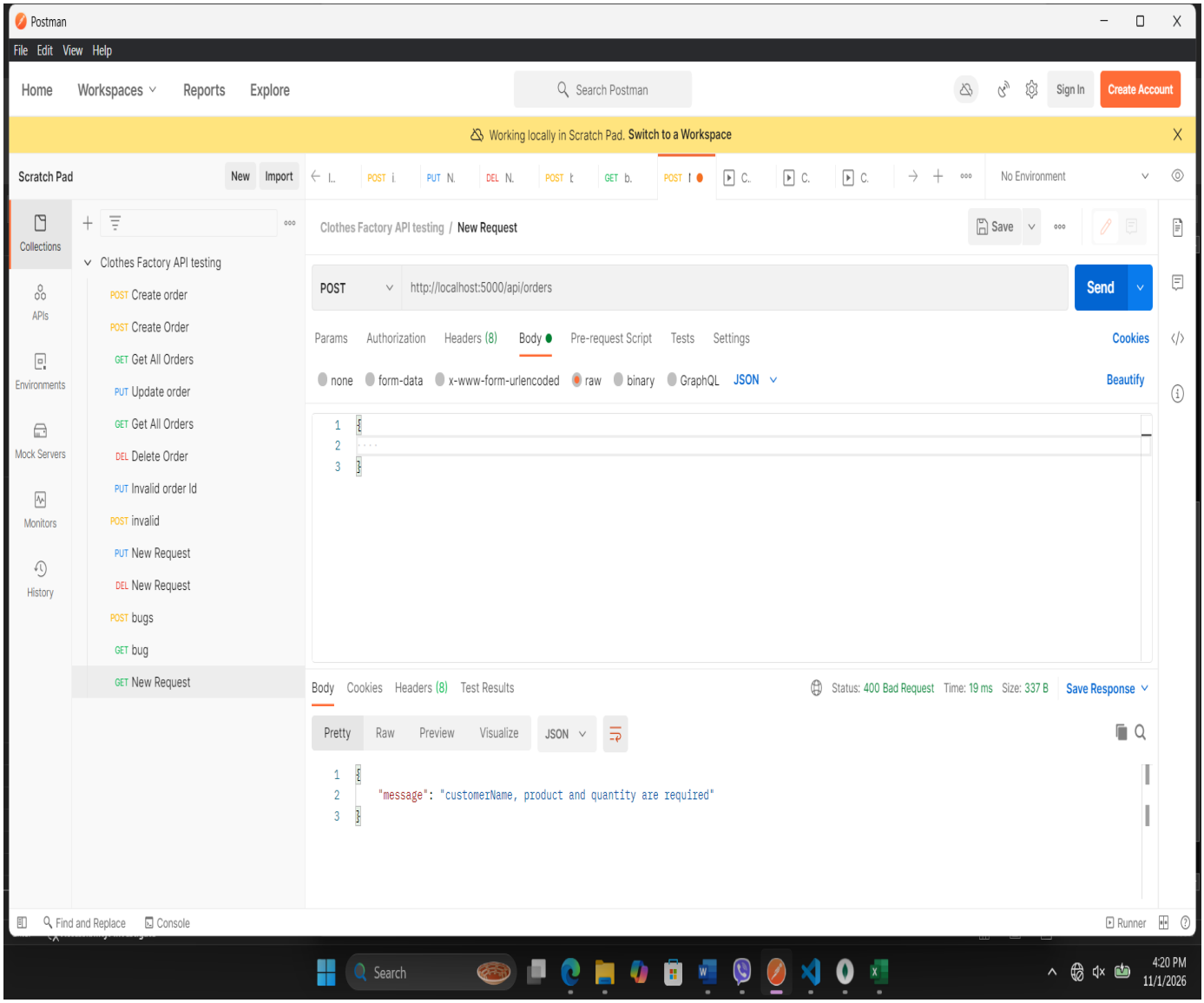
DELETE request evidence.



POST request evidence for negative quantity bug.



GET response showing the saved negative-quantity order.



POST request evidence for required-field validation.